

Lab 4

汤谨丞 2400012962

Task 1: Inverse Kinematics

1-1 Forward kinematic

认为该系统是一个链表结构，每个关节*i*只有一个父关节*i-1*和一个子关节*i+1*。

对于每一个关节，全局旋转量和全局位移量计算方式如下。注意，每一个关节的局部位移是在自己的局部坐标系下的，转化成全局位移需要考虑父关节的全局旋转。

```
ik.JointGlobalRotation[i] = ik.JointGlobalRotation[i - 1] * ik.JointLocalRotation[i];  
ik.JointGlobalPosition[i] = ik.JointGlobalPosition[i - 1] + ik.JointGlobalRotation[i - 1]  
* ik.JointLocalOffset[i];
```

1-2 CCD IK

从后向前遍历所有关节（从倒数第二个关节开始），对于每个关节*i*：

- 1.计算当前关节与末关节、当前关节与目标点的连线向量；
- 2.使用rotation函数算出将二者对齐的旋转量，更新当前关节的局部旋转
- 3.从关节*i*向后做前向运动学传播

使用CCD IK的效果如下

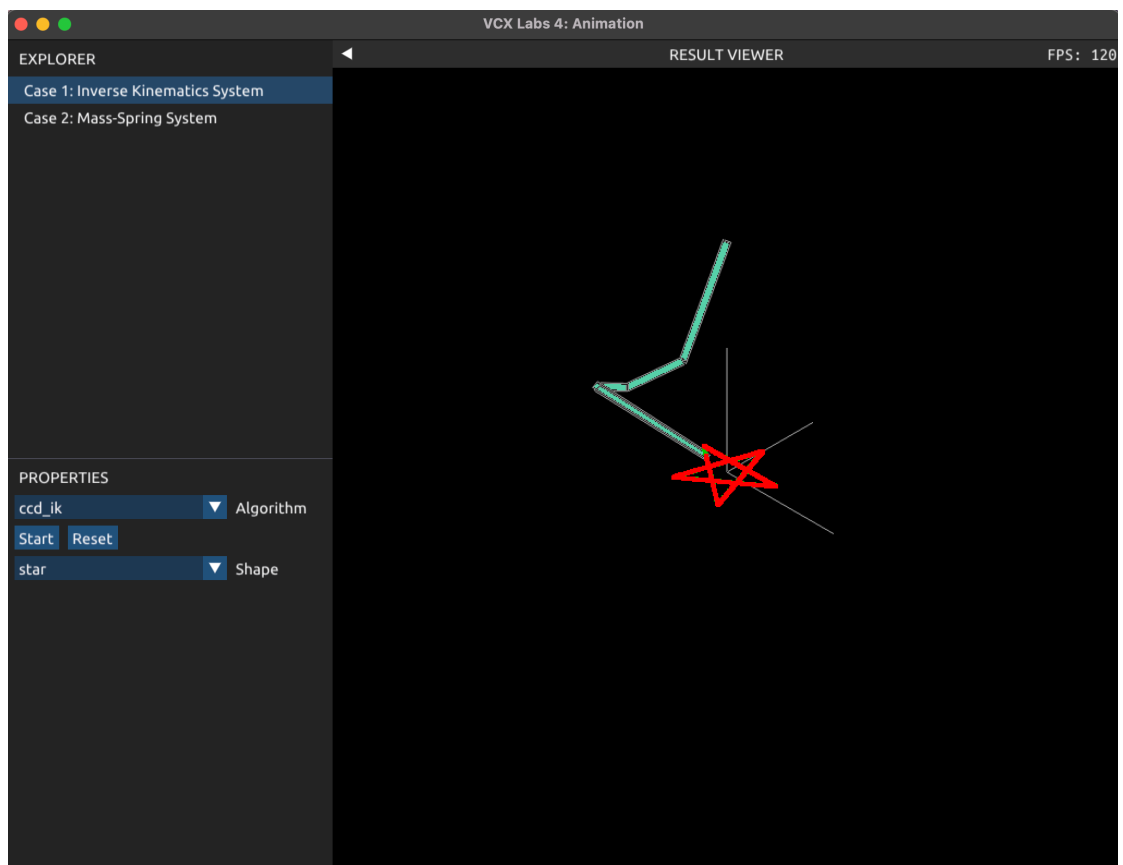


Figure 1

1-3 FABR IK

算法流程和图示如下

FABRIK n 个节点

① Backward

末端关节 $P_n \rightarrow$ 目标 t $P_n^{back=t}$

从后向前更新, 将 P_i 更新到 P_i^{back}

$i = n-1, \dots, 0$

$$|P_{i+1}^{back} - P_i^{back}| = |l_{i+1}| \quad \text{且} \quad \underbrace{P_i, P_i^{back}, P_{i+1}^{back}}_{\text{共线}}$$

解 P_i^{back}

$$P_i^{back} = P_{i+1}^{back} + \frac{P_i - P_{i+1}^{back}}{|P_i - P_{i+1}^{back}|} |l_{i+1}|$$

得到了一组 $\{P_i^{back}\}_{i=1}^n$ (板板不用)

方向

② Forward

$$P_0^{for} = P_0$$

从前向后遍历 将 P_i^{back} 更新到 P_i^{for}

$i = 1, 2, \dots, n$

$$|P_i^{for} - P_{i-1}^{for}| = |l_i| \quad \frac{P_i^{back} - P_{i-1}^{for}}{\sqrt{}} \quad \frac{P_{i-1}^{for}}{\sqrt{}} \text{共线}$$

$$P_i^{for} = P_{i-1}^{for} + \frac{P_i^{back} - P_{i-1}^{for}}{|P_i^{back} - P_{i-1}^{for}|} |l_i|$$

$$\{P_i^{for}\}_{i=1}^n$$

得到了一次结果

Figure 2-1

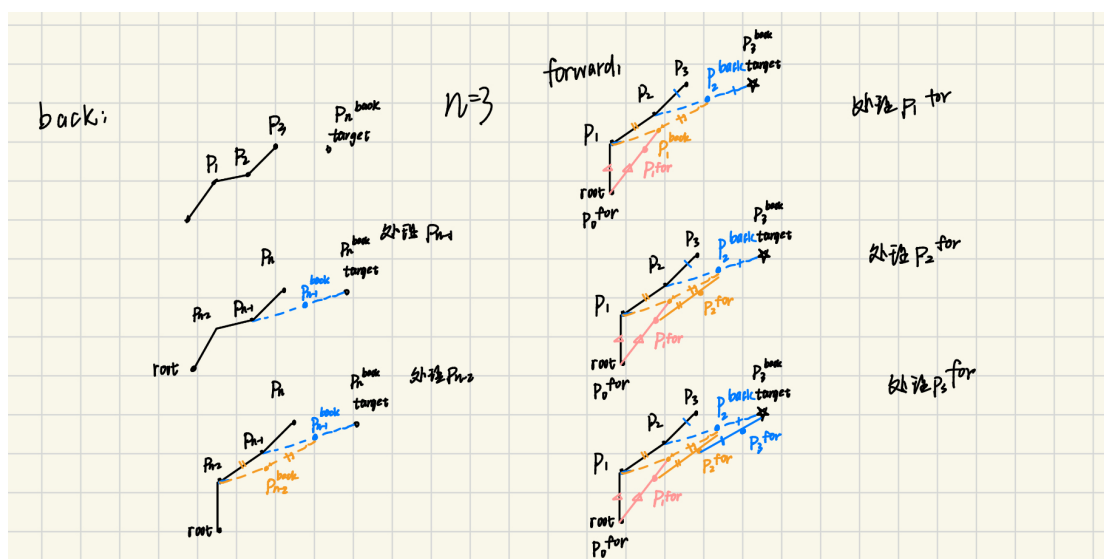


Figure 2-2

使用FABR IK的效果如下

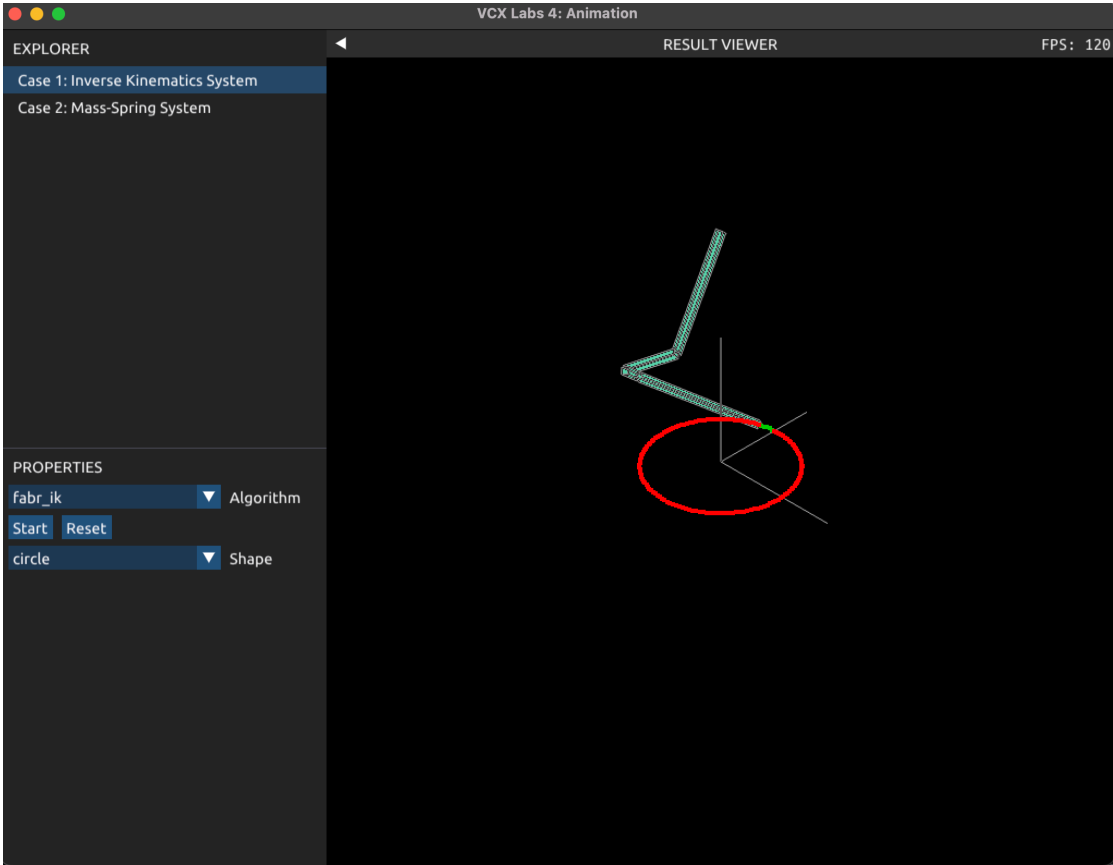


Figure 2-3

1-4 自定义曲线

仿照给出的BuildCustomTargetPosition函数，修改xval和yval的值，并用下列语句保存：

```
(*custom)[index++] = glm::vec3(x_val, 0.0f, y_val);
```

我绘制了我的姓名首字母TJC。T只需要横竖两条直线；J需要一条横线，一条竖线和一个半圆；C则是取了一个圆的一部分。效果如下。

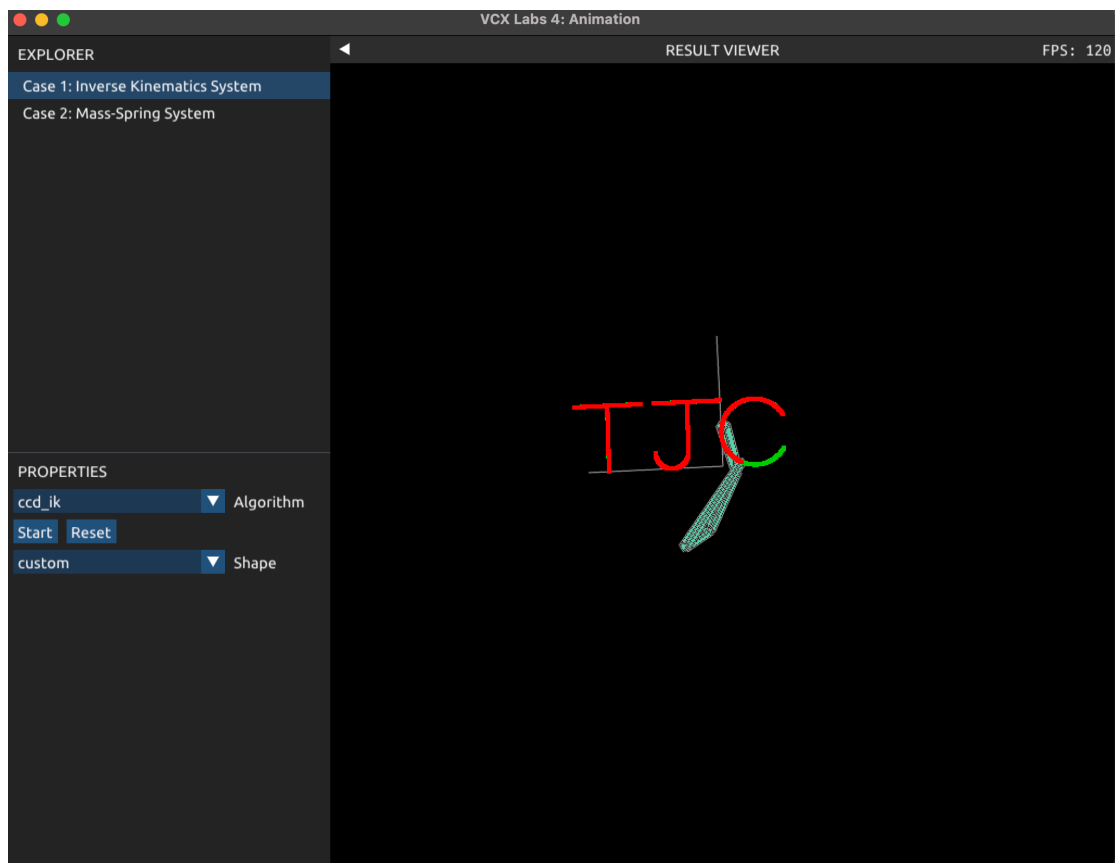


Figure 3

Bonus task:

让采样点更均匀的方法：计算本次采样点和上次采样点的距离，如果距离超过给定的阈值 $len_bar=0.02$ 就进行跨度为 len_bar 的重采样。采用与不采用该方法的对比如下。

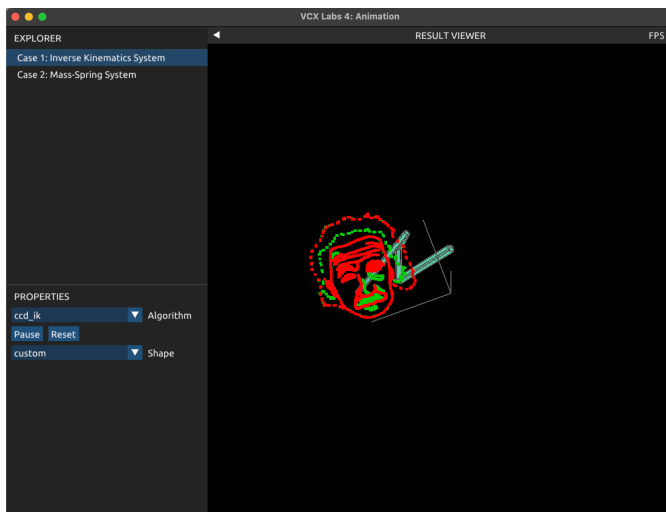


Figure 4: 无优化，头发稀疏

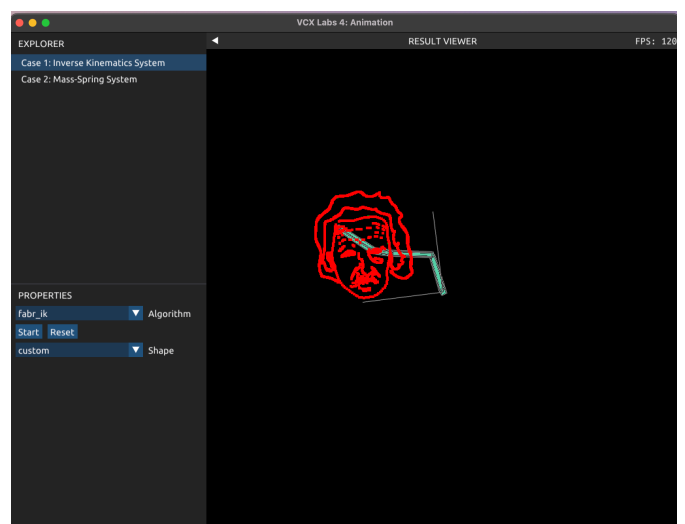


Figure 5: 有优化，头发均匀

回答问题：

1.如果目标位置太远，无法到达，IK 结果会怎样？

IK 会把末端尽可能伸向目标，但停在机械臂能到达的最远点，例如机械臂完全伸直变成一条直线。

2.比较 CCD IK 和 FABR IK 所需要的迭代次数

FABR 的迭代次数更少。CCD IK更新是逐关节改善末端位置，而FABR IK直接移动整体关节坐标。FABR的收敛速度显著快于CCD，对于长链条更加明显。

3.由于 IK 是多解问题，在个别情况下，会出现前后两帧关节旋转抖动的情况。怎样避免或是缓解这种情况？

添加阻尼项，限制关节频繁跳跃；限制每个关节最大的旋转变化量；多解时，优先选择与上一帧接近的解。

Task 2: Mass-Spring System

实现思路

1.计算内力 f_{ij} 和 f_{ji} ，加到对应点的内力上，得到forces向量即为E的负梯度

遍历所有弹簧，得到起点 p_0 ,终点 p_1 ，计算弹力，累加到force数组。

```
glm::vec3 const x01 = system.Positions[p1] - system.Positions[p0];
glm::vec3 const e01 = glm::normalize(x01);
glm::vec3 f = system.Stiffness * (glm::length(x01) - spring.RestLength) * e01; //无需阻尼项
forces[p0] += f;
forces[p1] -= f;
```

2.计算E的Hessian矩阵。

这是一个 $3n \times 3n$ 的矩阵，将其划分成 $n \times n$ 个 3×3 的子矩阵，位置 $(i,i)(j,j)$ 的子矩阵为 H_{ij} ;位置 $(i,j)(j,i)$ 的子矩阵为 $-H_{ij}$ 。遍历所有弹簧，起点为 p_0 ,终点为 p_1

```
glm::vec3 const x10 = system.Positions[p0] - system.Positions[p1];
glm::vec3 const e01 = glm::normalize(x10);
float l_ij = glm::length(x10);
glm::mat3 H_ij = system.Stiffness * ( glm::outerProduct(e01, e01) + (1 -
spring.RestLength / l_ij) * (glm::mat3(1.0f) - glm::outerProduct(e01, e01)) / l_ij );
```

3.计算 y_{tk}

$$\vec{y}(t_k) = \vec{x}(t_k) + \Delta t \vec{v}(t_k) + (\Delta t)^2 M^{-1} \vec{f}_{\text{ext}} \quad (1)$$

其中，认为外力全部来自于重力。

```
gravity.setZero();
for(int i=0;i<n;i++)
    gravity[3*i+1]=-system.Gravity;
Eigen::VectorXf y=x + dt*v + dt*dt*gravity;
```

4.计算g的梯度和Hessian矩阵

```

//构建M_diag
Eigen::SparseMatrix<float> M_diag(n * 3, n * 3);
for (int i = 0; i < n * 3; i++)
    M_diag.insert(i, i) = system.Mass;
//计算E的梯度
Eigen::VectorXf E_grad = -glm2eigen(forces);
//计算g的梯度
Eigen::VectorXf g_grad = 1/(dt*dt)*M_diag*(x-y)+E_grad;
//计算g的Hessian矩阵
Eigen::SparseMatrix<float> g_Hessian = 1/(dt*dt)*M_diag + H_E;

```

5.调用ComputeSimplicialLLT函数求解线性方程组得到delta_x

$$[H_g(\vec{x}(t_k))](\vec{x}(t_{k+1}) - \vec{x}(t_k)) = -\nabla g(\vec{x}(t_k)) \quad (2)$$

6.更新位置和速度。

对于固定的点，不用更新。

实现效果

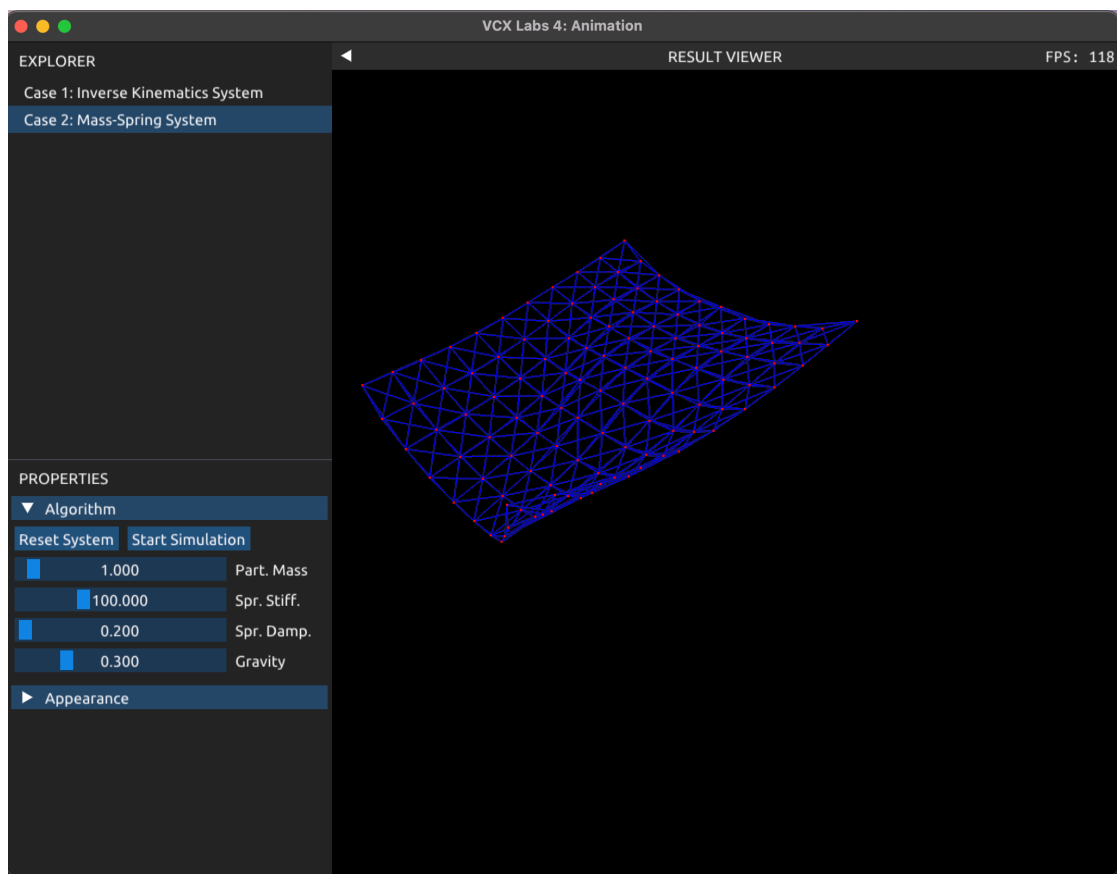


Figure 6